UNIVERSITÉ PARIS SACLAY
Master E3A – SETI

T1 Architecture des processeurs
TD2 Hiérarchie mémoire

Auteur : Alain Mérigot (Université Paris Saclay)

1 Étiquettes et index de cache
Corrections faites en séances

2 Caches données
Corrections faites en séances

3 Caches instructions
Un processeur a un jeu d'instructions RISC, avec des instructions de longueur fixe d'un mot. Il a un cache instructions de 2 Kmots, avec des lignes de 8 mots. Il utilise la correspondance directe. Il exécute le code de la Figure 3, constitué de deux boucles imbriquées. Les seuls branchements du programme sont les deux branchements de boucle, aux adresses 239 et 1200.
Le temps pour un succès cache est t et un défaut de cache coûte $8t$.

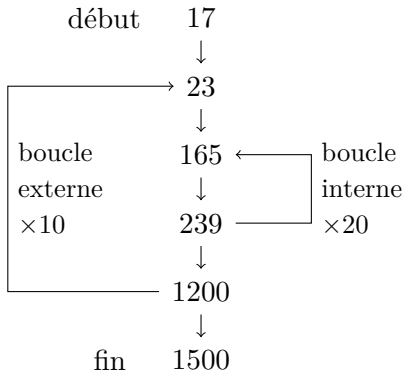


FIGURE 1 – Programme

3.1 En négligeant l'effet des défauts de caches pour les données, quel est le temps d'exécution du programme de la Figure 3.

Rep : *Le cache comprend 256 lignes et chaque ligne contient 8 instructions.*
La première instruction est à l'adresse 17 et dans la ligne 2 du cache.
La dernière est à l'adresse 1500 et dans la ligne 187 du cache.
Il n'y a aucun conflit. Défauts uniquement au démarrage. : 186 défauts de cache.

Le nombre d'instructions exécuté est :
17 à 22 et 1201 à 1500 exécutées 1 fois : 306
*23 à 164 et 240 à 1200 exécutées 10 fois, soit (142+961)*10 =11030 instructions*
*165 à 239 exécutées 10*20 fois, soit 75*200 instruction = 15000*

*T exécution : (306+11030+15000)t+186*7t=27638t*

3.2 Reprendre la question précédente en supposant un cache de 1 Kmots avec correspondance directe, puis l'associativité 2 voies (2 lignes par ensemble)

Rep : *Dans le cas d'un cache de 1k mot, tout le programme ne tient pas dans le cache (128 lignes de 8 instr). Le programme utilise pour les instructions 17-1023 les lignes 2-127 et pour les instructions 1024-1500 les lignes 1 à 59. Conflit sur les lignes 2 à 59.*
Analyse des défauts de cache :
Un défaut lors du premier passage pour chacun des blocs
Pas de défaut dans la boucle interne

*Lors du 2ème passage dans la boucle externe, les blocs 2 à 22 (correspond au instructions 1040-1200) auront remplacé des blocs existants (des instructions 23-183). 2*21 défaut par itération → 21*2*9=378 défauts.*
Total : 186+378=564 défauts
*Temps : 16336t+564*8t=30848T*

Cache associatif 2 voies de 1k mots => 64 blocs dans le cache (de 2 lignes chacun) Les conflits sont plus nombreux. Notamment les blocs 2 à 22 seront utilisés par les instructions d'adresse 23 à 183, 528 à 695 et 1040 à 1207.
Défaut lors du premier accès à chaque ligne d'instructions.
*Deuxième itération de la boucle externe (et suivante). Pour les blocs 2 à 22, a chaque nouvel accès, il y aura un défaut de cache du fait du mécanisme LRU qui enlève la ligne la plus ancienne. 3 défauts/itération pour chacune des lignes 2 à 22, soit 21*3*9 défauts (567) Soit un total de défauts de 186+567=753*
*Temps d'exécution : 26336t+753*8t=32360*

4 Mémoire virtuelle et TLB

Soit le code suivant de multiplication de matrices.

```
1  float a[1024][1024], b[1024][1024], c[1024][1024];
2  multiply()
3  {
4    int i, j, k;
5    for(i = 0; i < 1024; i++)
6      for(j = 0; j < 1024; j++){
7        c[i][j]=0.0;
8        for(k = 0; k < 1024; k++)
9          c[i][j] += a[i][k] * b[k][j];
10     }
11 }
```

On suppose que le code binaire pour exécuter cette fonction tient dans une page de 4 Ko et que la pile tient elle aussi dans une page.

4.1 Quel est le nombre de défauts de TLB avec un TLB de 8 entrées utilisant LRU comme politique de remplacement ?.

NB : Les pages pour le code binaire et pour la pile seront en permanence en mémoire, et les entrées correspondantes du TLB seront donc constamment utilisées. 6 entrées du TLB restent disponibles pour l'exécution du programme.

Rep : *Au sein de la boucle interne, pour exécuter*
`c[i][j] += a[i][k] * b[k][j];`
il faut 3 entrées de TLB valides pour `a`, `b` *et* `c`. *Chaque entrée de TLB permet d'adresser 4ko, donc 1k* `floats` *(et donc une ligne de chaque matrice).*

Défauts de TLB :
ligne 7 sur `c` *pour chaque nouvelle valeur* `i` *(1k TLB miss)*
ligne 9 sur `a` *idéalement un TLB miss par nouvelle valeur de* `i`. *Comme* `a` *est utilisé systématiquement, le mécanisme LRU assure que l'entrée de TLB va rester pour les valeurs successives* `j`. *(1k TLB miss)*
ligne 9 sur `b` *pour chaque nouvelle valeur de* `k` *(1k*1k*1k TLB miss !!)*
Soit un total de 1024(2+1024*1024)=1073743872 défauts de TLB*

4.2 Reprendre la question précédente en utilisant l'algorithme ikj (optionnel)

```
1 float a[1024][1024], b[1024][1024], c[1024][1024];
2 multiply_ikj()
3 {
4   int i, j, k;
5   for(i = 0; i < 1024; i++)
6     for(k = 0; k < 1024; k++){
7       float aa = a[i][k];
8       for(j = 0; j < 1024; j++)
9         c[i][j] += aa * b[k][j];
10    }
11 }
```

Rep : *Ligne 7 `a[i][k]` va faire un défaut de TLB pour chaque nouvelle valeur de `i`. (1k defaut)*
*Pour la ligne 9, `b[k][j]` va faire un défaut de TLB pour chaque nouvelle valeur de `k` (1k*1k défauts)*
`c[i][j]` va faire un défaut pour chaque nouvelle valeur de `i` (1k défauts).
Soit un total de 1024(2+1024)=1050624 défauts de TLB (1000 fois inférieur au cas précédent !).*